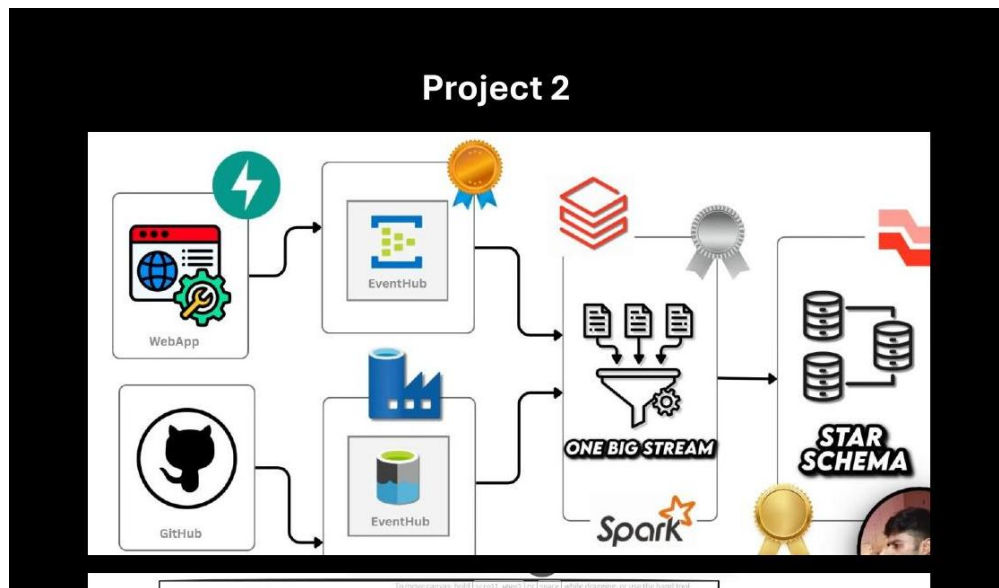




Uber Data Engineering Project

End-to-End Azure Data Pipeline

Azure Event Hubs • Azure Data Factory • Azure Databricks
Bronze → Silver → Gold • Medallion Architecture • Star Schema



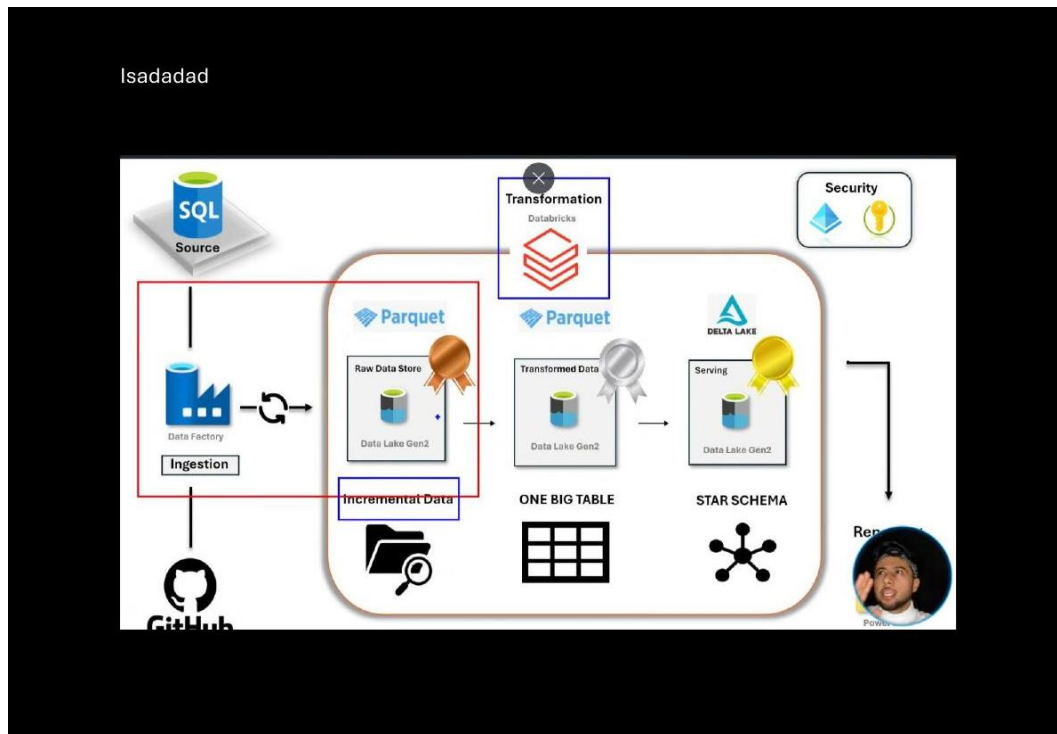
Full Project Architecture — WebApp + GitHub → EventHub + ADF → Spark DLT → Star Schema

GitHub: [ruchika123yadav/Uber_DE_Project](https://github.com/ruchika123yadav/Uber_DE_Project)
Azure Cloud | PySpark | DLT | Jinja2 | Star Schema

Project Overview

This project builds a complete, production-grade data engineering pipeline on Microsoft Azure for Uber ride data. It ingests real-time events from a web application, processes historical bulk data from GitHub, and transforms everything through three data quality layers — Bronze, Silver, and Gold — ultimately producing a Star Schema model ready for analytics and reporting.

Full Lakehouse Architecture



Lakehouse Architecture: Data Factory (Ingestion) → ADLS Gen2 (Raw Parquet) → Databricks (Transformation) → Star Schema (Serving)

Azure Resources Created

Azure Service	Resource Name / Purpose
Storage Account	Container: githubdata — ADLS Gen2 data lake
Event Hubs Namespace	uber-eventhub-pro
Event Hub Topic	ubertopic — receives ride booking events
Azure Data Factory	Pipeline: httpToStorageAccount — GitHub → ADLS
Databricks Catalog	uber_project Schema: bronze_layer Volume: uber_volume

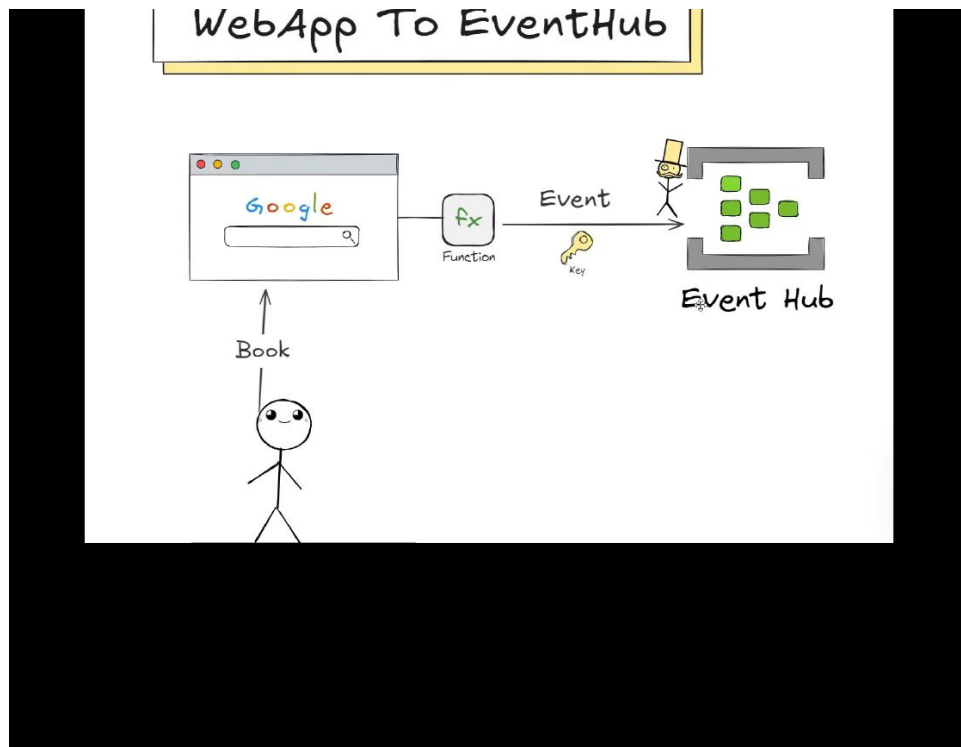
End-to-End Layer Summary

Layer	Tool	Output
 Ingestion	Event Hubs + ADF	rides_raw + ADLS files
 Bronze	Databricks DLT	rides_raw streaming table
 Silver	Databricks DLT	stg_rides + OBT (Jinja joins)
 Gold	Databricks DLT	Star Schema (Dims + Fact + SCD)

⚡ Part 1: Event Hub — Real-Time Ingestion

When a user clicks 'Book a Ride' in the web app, ride data is instantly sent to Azure Event Hubs. This is the real-time ingestion layer of the pipeline.

🎮 Flow Diagram: WebApp → Event Hub



User clicks 'Book' → Function triggers with Key → Event sent to Event Hub (uber-eventhub-pro / ubertopic)

📌 What Was Built

- Created Event Hub Namespace: uber-eventhub-pro
- Created Event Hub Topic: ubertopic (1 partition, 168-hour retention)
- Added Shared Access Policies: Send + Listen
- Built Web App with 'Book the Uber Ride' button
- Connection via connection.py — connection string + event hub name from .env
- Every button click sends a ride event directly to ubertopic

💻 Azure Portal: Creating the Event Hub (ubertopic)

Namespace. If you need more power, you just move a slider in the Azure Portal to increase TUs, and Azure handles the underlying hardware scaling instantly.

Home

Create Event Hub

Event Hubs

Basics Capture Review + create

Event Hub Details

Enter required settings for this event hub, including partition count and message retention.

Name *

Partition count

Retention

Configure retention settings for this Event Hub. [Learn more](#)

Cleanup policy

Retention time (hrs) * min. 1 hour, max. 168 hours (7days)

Azure Portal: Create Event Hub — Name: ubertopic, Partition: 1, Retention: 168 hrs (7 days)

Event Hub Concepts (Interview-Ready)

Pub-Sub vs Producer-Consumer vs Event Hubs

Model	Mechanism	Azure Tool
Pub-Sub	Publisher sends to Topic; multiple subscribers read independently	Service Bus / Event Grid
Producer-Consumer	Producer writes to queue; one consumer processes and it's gone	Azure Storage Queues
Event Hubs	Tape recorder — events stored, multiple consumers read at own pace	Azure Event Hubs ☒

Event Hub vs Apache Kafka

Feature	Apache Kafka (Self-managed)	Azure Event Hubs (PaaS)
Infrastructure	You manage cluster, VM, RAM, storage	Zero infra — Azure handles it
Setup Time	Days to configure HA cluster	Seconds via Azure Portal
Scaling	Manually add Brokers	Auto-Inflate scales TUs automatically
Maintenance	You patch OS, Kafka versions, hardware	Fully managed by Microsoft
Migration	N/A	Kafka-compatible endpoint — just change connection string!

💡 **Key Insight: Event Hubs biggest advantage = PaaS. No infra headaches. In Kafka you worry about cluster size, machine size, VM size. Azure handles ALL of that.**

Namespace vs Event Hub Entity

Concept	Description
Event Hub Namespace	Logical container for all Event Hubs. Sets Pricing Tier, Region, Throughput Units.
Event Hub (Topic)	Actual data stream. Child inside Namespace. You set Partitions & Retention here.
Throughput Units	Set at Namespace level. All hubs share TUs — overuse causes throttling.
Partitions	Physical lanes — more lanes = more parallel consumers. Same partition = ordered data.
Consumer Group	A 'view' of the hub. Different apps read same stream independently at own offset.

Interview Q&A — Event Hubs

Q: When use Event Hubs vs Service Bus?

Event Hubs: High-volume telemetry/streaming (millions of events/sec) — IoT, clickstreams

Service Bus: Transactional messaging needing sessions, dead-letter, ordering guarantees

Q: What is Auto-Inflate?

Automatically increases Throughput Units when traffic exceeds limit.

Prevents pipeline throttling without manual intervention.

Q: Why is Event Hubs called 'Distributed'?

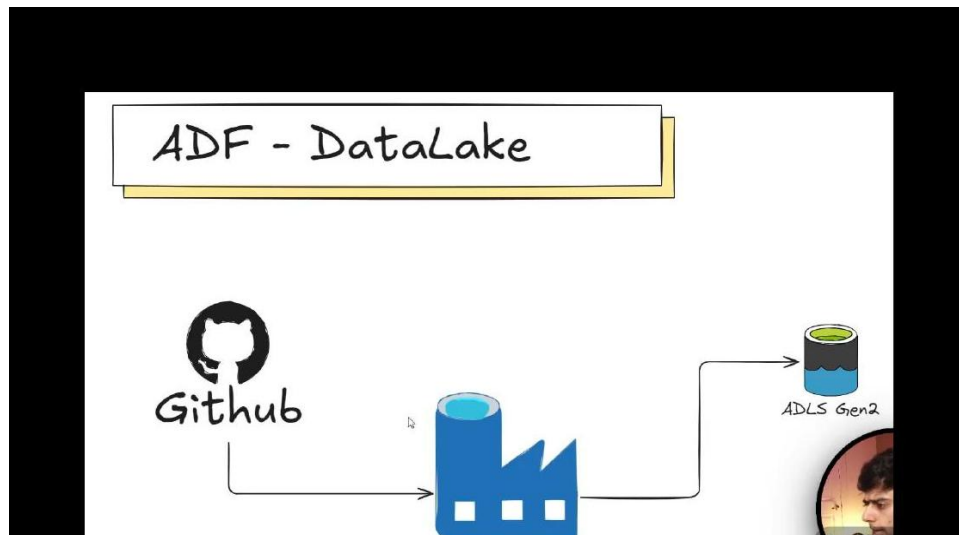
Uses Partitions — data spread across multiple storage units.

Multiple consumers read different partitions simultaneously → high speed and scale.

Part 2: Azure Data Factory — Batch Ingestion

Azure Data Factory (ADF) pulls historical Uber data files from GitHub and loads them into Azure Data Lake Storage (ADLS Gen2) using a metadata-driven pipeline.

Architecture: GitHub → ADF → ADLS Gen2



ADF Pipeline: GitHub (HTTP Linked Service) → Azure Data Factory → ADLS Gen2 (githubdata container)

Source Files on GitHub

File	Description
bulk_rides	Main Uber rides dataset — large historical data (main fact data)
map_cities	Mapping table — city codes to city names
map_cancellation	Mapping table — cancellation reason codes
Other map_* files	Various lookup tables that enrich event hub data with context

Why mapping files? Because ride events from Event Hubs contain IDs (not names). Mapping files provide context — e.g., city_id 101 = 'Hyderabad'. They are static reference files that enrich streaming data.

Pipeline: httpToStorageAccount

Pattern: Metadata-Driven Pipeline — Write once, reuse forever. Add new files by updating config only — no code changes needed.

How the Metadata-Driven Pipeline Works

Step-by-step Pipeline Flow
Step 1 — Lookup Activity: Reads list of all file names from metadata/config source
Step 2 — ForEach Loop: Iterates over each file name returned by Lookup
Step 3 — Copy Activity (inside loop): For each file → copies from GitHub (HTTP) to ADLS Gen2
Step 4 — Parameters in Datasets: File name passed as parameter — zero hardcoding
Result: All files (bulk_rides + all map_* files) land in the 'githubdata' container

Linked Service Choice: HTTP

- Used HTTP linked service (not REST or GitHub connector)
- REST returns JSON — HTTP returns raw file content, which is what we need
- HTTP = used whenever making a direct API/file call to an external URL

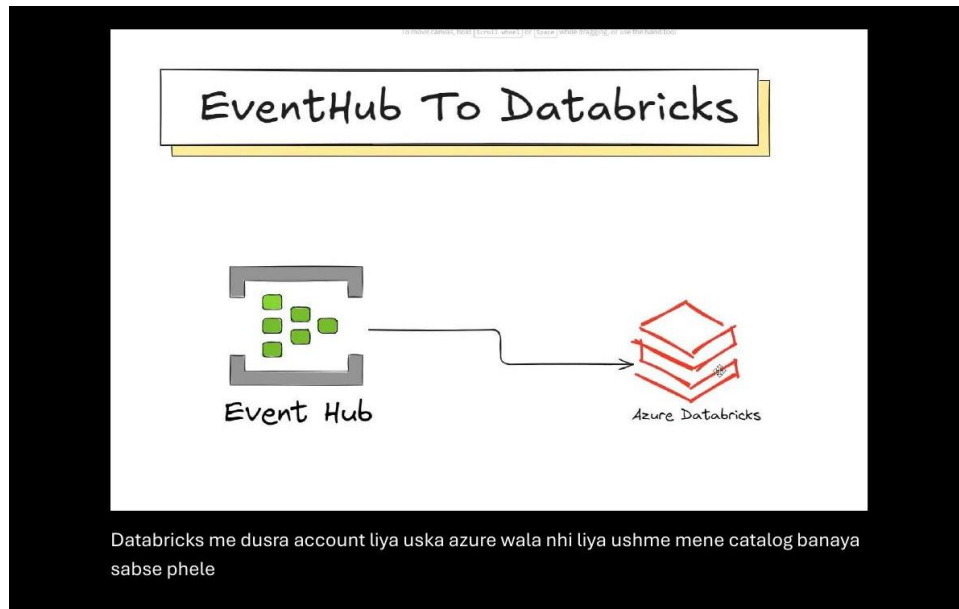
Metadata-Driven Design Principle

Why Metadata-Driven?
WITHOUT it: Add new file → modify pipeline code → redeploy → risky and time-consuming
WITH it: Add new file → update config file → pipeline auto-handles it — zero code change
Write code ONCE. Reuse it MULTIPLE TIMES.
Same principle as used later in Databricks with Jinja2 templates

💡 Part 3: Azure Databricks — Transformation

Databricks is where all transformation happens — connecting to Event Hubs and ADLS, processing data through Bronze → Silver → Gold layers using Delta Live Tables (DLT), and building the Star Schema.

🔗 EventHub → Databricks Connection



Event Hub (uber-eventhub-pro) connected to Azure Databricks using the Listen shared access policy

📁 Databricks Setup

Object	Name
Catalog	uber_project
Schema (Bronze)	bronze_layer
Volume	uber_volume
DLT Pipeline	uber_data_ingestion

⚡ What Are Spark Declarative Pipelines (DLT)?

Traditional Spark is Imperative — you write every step. Delta Live Tables is Declarative — define WHAT you want, Spark handles HOW.

Feature	Standard Spark (Imperative)	Delta Live Tables (Declarative)
Code Style	df.write.save()	@dlt.table / CREATE LIVE TABLE
Data Quality	Manual if/else filter checks	Built-in Expectations (RETAIN/DROP/FAIL)
Recovery	Manual checkpoint paths	Automatic checkpoint management
Infrastructure	You manage cluster	DLT manages Enhanced Autoscaling
Lineage	Hard to track manually	Visual DAG in UI — automatic
Dependencies	Trigger notebooks manually in order	DLT builds DAG — auto-handles order

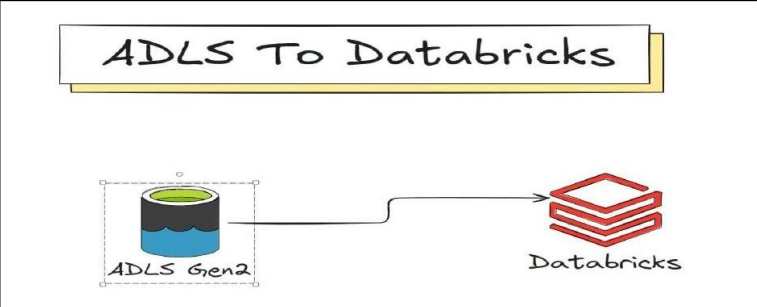
💡 **Pizza Analogy: Imperative = giving someone a recipe and watching every step. Declarative = ordering a pizza — you say what you want, the kitchen handles the oven and timing.**

🔗 Connecting ADLS Gen2 to Databricks

injection?

- **Answer:** No. Community Edition is a shared, multi-tenant environment managed by Databricks on AWS. VNet injection is only available in the paid Azure Databricks tiers.

DATABRICKS:



The diagram shows a box labeled 'ADLS To Databricks'. Inside the box, on the left, is an icon for 'ADLS Gen2' (a blue cylinder with a green top). An arrow points from this icon to an icon for 'Databricks' (a red 3D stack of blocks) on the right.

ADLS Gen2 (githubdata container) connected to Databricks Community Edition using SAS Token

Why SAS Token (not Access Key)?

Connection Method: SAS Token
Used Databricks Community Edition (hosted on AWS, not Azure) — Managed Identity unavailable
SAS Token = Shared Access Signature = temporary token with expiry + limited permissions

Safer than Access Key — scope limited to specific container and specific time window

Created SAS token at container level in Azure Portal → used in Databricks notebooks

```
# Connect ADLS Gen2 to Databricks using SAS Token
spark.conf.set(
    "fs.azure.account.sastoken.<storage-account>.dfs.core.windows.net/<container>",
    "<your-sas-token>"
)

# Read bulk rides data
df = spark.read.csv("abfss://githubdata@<storage-account>.dfs.core.windows.net/bulk_rides.csv")
```

PySpark vs Pandas — When to use which?

Framework	Use Case
PySpark DataFrame	Large datasets — fact tables, streaming rides, real-time data. Big tables need distributed compute.
Pandas DataFrame	Small static mapping files — map_cities, map_cancellation etc. Also needed for direct SAS token reads.

Bronze Layer — Raw Data Ingestion

The Bronze layer stores raw, unmodified data exactly as received — Event Hubs (streaming) and ADLS (batch/historical). No business transformations happen here.

Source 1: Event Hub → rides_raw

Event Hub → Databricks DLT (rides_raw)
Connected Databricks to 'uber-eventhub-pro' using the Listen policy (connection string)
Created DLT Streaming Live Table: rides_raw — stores all incoming real-time ride events
Used @dlt decorator (declarative pipeline function) to define the table
Table type: Streaming Live Table — only processes NEW events since last run

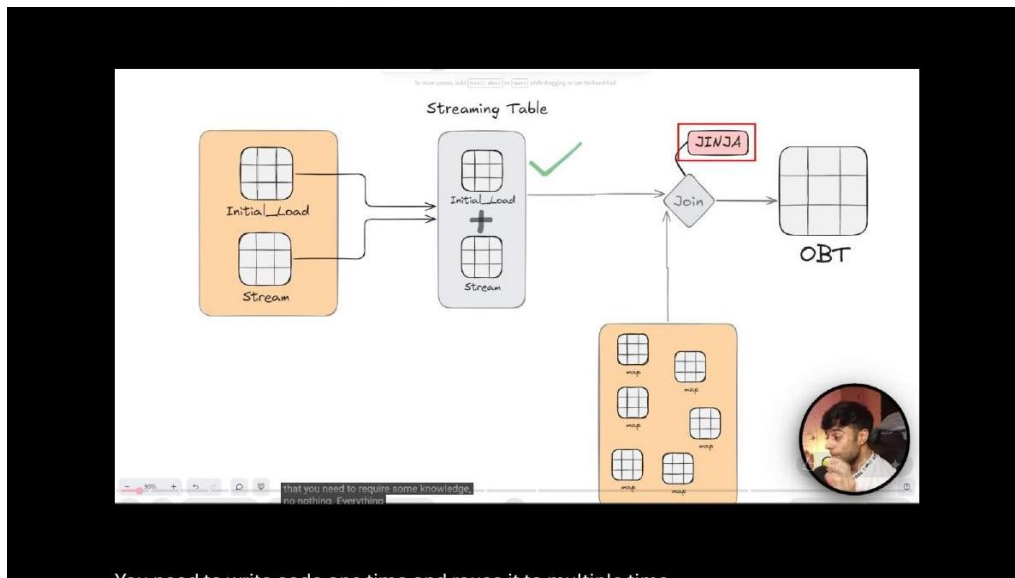
Source 2: ADLS bulk_rides → Bronze

- Connected ADLS Gen2 container to Databricks via SAS token
- Loaded bulk_rides.csv (historical Uber data) from githubdata container using PySpark
- Loaded all mapping files (map_cities, map_cancellation etc.) using Pandas DataFrames
- Stored both in bronze_layer schema of uber_project catalog

○ Silver Layer — Cleaned & Merged Data

Silver layer takes bronze raw data, cleans it, resolves schema issues, merges streaming and historical data, and joins all mapping tables to produce a clean enriched dataset.

🎮 Silver Layer Architecture — Streaming Table + OBT



Initial_Load + Stream → merged into Streaming Table (stg_rides) → Jinja JOIN with mapping tables → OBT (One Big Table)

🔍 The Challenge: Merging Stream + Historical Data

Problem: If you union 100GB historical + 10MB stream every 5 mins, Spark re-reads ALL 100GB each time. Wastes compute, costs money, and causes duplication on failure.

Solution: Streaming Table (Stateful Bookmark)

Why Streaming Table solves the merge problem

1. Push 100GB historical data (Initial Load) once → Streaming Engine marks it 'Processed'
2. New 10MB stream arrives → Engine checks Checkpoint: only touch NEW 10MB
3. Merges incrementally — does NOT re-scan old data on each run
4. If pipeline fails → resumes from exact checkpoint — no duplication, no data loss

Result: Efficient, fault-tolerant incremental merge of history + real-time data

stg_rides Table — The Merge

- Schema defined explicitly for rides_raw (streaming) data — fixed data types
- JSON column inside rides_raw parsed to proper struct schema
- Streaming Table stg_rides = Initial Load + Stream (merged via DLT append)
- Both datasets aligned to same schema before merge

OBT — One Big Table (Jinja2 Joins)

After stg_rides is ready, it gets joined with ALL mapping tables to produce the One Big Table (OBT). Jinja2 template dynamically generates the SQL join query.

Actual Jinja2 Code from Databricks Notebook

```

12:00 PM (10) 1
import jinja2
from jinja2 import Template

2 minutes ago (10) 2
jinja_config=[
{
    "table": "uber_project.bronze_layer.stg_rides",
    "select": "uber_project.bronze_layer.stg_rides.*",
    "where": ""
},
{
    "table": "uber_project.bronze_layer.map_vehicle_makes map_vehicle_makes",
    "select": "map_vehicle_makes.vehicle_make",
    "where": "",
    "on": "stg_rides.vehicle_make_id = map_vehicle_makes.vehicle_make_id"
},
{
    "table": "uber_project.bronze_layer.map_vehicle_types map_vehicle_types",
    "select": "map_vehicle_types.vehicle_type,map_vehicle_types.description,map_vehicle_types.base_rate,map_vehicle_types.per_mile,map_vehicle_types.per_minute",
    "where": "",
    "on": "stg_rides.vehicle_type_id = map_vehicle_types.vehicle_type_id"
}
]

2 minutes ago (10) 3
jinja_str="""
select
    {% for config in jinja_config %}
    {{config.select}}
    {% if not loop.last %}
    *
    {% endif %}
    {% endfor %}

from
    {% for config in jinja_config %}
    {% if loop.first %}
    {{config.table}}
    {% else %}
    left join {{config.table}} on {{config.on}}
    {% endif %}
    {% endfor %}

{% for config in jinja_config %}

    {% if loop.first %}
    {% if config.where != "" %}
        where
        {{config.where}}
    {% endif %}
    {{config.where}}
    {% if not loop.last %}
    {% if config.where != "" %}
        and
        {{config.where}}
    {% endif %}
    {% endif %}
    {% endfor %}

"""

template=Template(jinja_str)
rendered_template=template.render(jinja_config=jinja_config)
print(rendered_template)

```

Databricks Notebook: `jinja_config` defines all tables + joins; Jinja template renders the full SQL query dynamically

Why Jinja2 Template?

Without Jinja

With Jinja

Write a new JOIN query every time a table is added	Just add the table to jinja_config list
Risk of errors in repetitive SQL	Template generates correct SQL automatically
Hard to maintain at scale	Write once, reuse forever — same as metadata-driven ADF pipeline

Streaming Joins & Watermarking

OBT joins were done as streaming joins (not regular batch joins). This required Watermarking to manage memory.

What is Watermarking?
Problem: Streaming data arrives out of order (phone offline → event delayed)
Watermark = (Max Event Time Seen) - (Allowed Lateness Delay)
Without it: Spark holds ALL state in memory forever → OutOfMemory (OOM) crash
With it: Once watermark passes a window, Spark drops that state — cluster stays healthy
In DLT SQL: Use STREAM keyword before table name (replaces readStream from PySpark)

```
-- DLT SQL Streaming Join with Watermark
SELECT stg_rides.*, map_vehicle_makes.vehicle_make
FROM
  STREAM(uber_project.bronze_layer.stg_rides)
  WATERMARK booking_timestamp DELAY OF INTERVAL 3 MINUTES stg_rides
LEFT JOIN map_vehicle_makes
  ON stg_rides.vehicle_make_id = map_vehicle_makes.vehicle_make_id
```

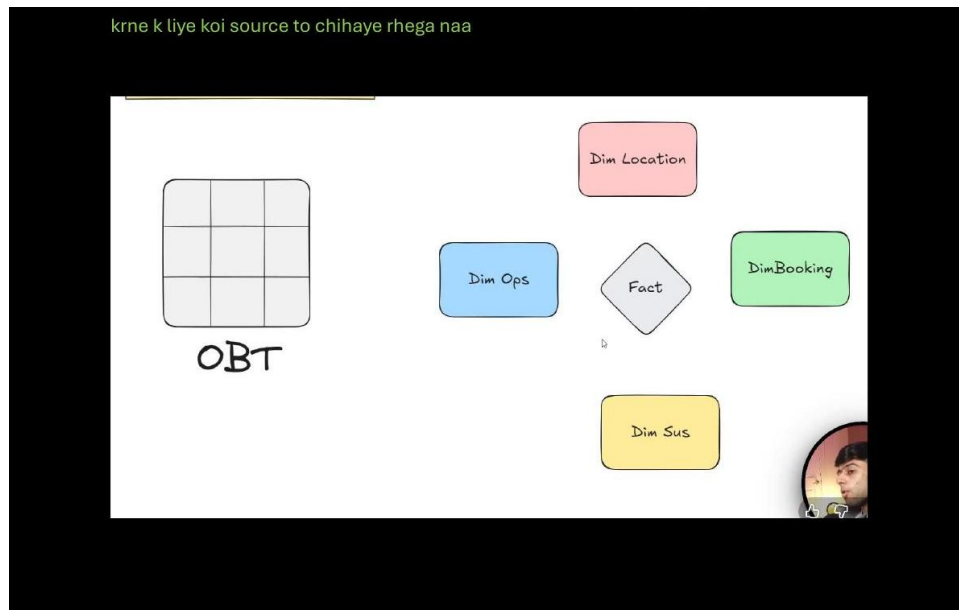
Slowly Changing Dimension (SCD) Setup in Silver

- Added updated_at column to map_cities table
- Re-ran ADF pipeline to reload the updated map_cities into ADLS
- Re-uploaded to Databricks and re-ran the silver DLT pipeline (stg_rides)
- This sets up the SCD Type 2 tracking used later in Gold layer's dimLocation

● Gold Layer — Star Schema (Data Modelling)

Gold layer takes the One Big Table (OBT) and performs data modelling — splitting into Dimension tables and a Fact table in a Star Schema, optimized for analytics and reporting.

★ OBT → Star Schema Diagram



Data Modelling: OBT broken into Dim Location, Dim Ops, Dim Booking, Dim Sus + central Fact table (Star Schema)

📊 Star Schema Design

Table	Description
dimLocation	Location dimension — SCD Type 2 (tracks historical changes via updated_at)
dimBooking	Booking type dimension
dimPassenger	Passenger dimension — built using a View as CDC source
dimDriver	Driver dimension
dimPayment	Payment method dimension
FactRides	Central fact table — measurable metrics + FKs to all dimensions

🔍 Slowly Changing Dimension (SCD) — dimLocation

What is SCD Type 2?

Problem: What if a city's data changes? (e.g., region re-classification)
SCD Type 2 tracks HISTORY — old record kept with end date, new record inserted
Implementation: added updated_at column to map_cities → ADF reloaded the file
Databricks Auto CDC feature used to apply SCD logic to dimLocation table
Result: Historical changes to location data preserved — full audit trail

dimPassenger — CDC View Pattern

A View was created to act as the CDC source for dimPassenger. Auto CDC requires a source table/view to hold incoming data before applying changes to the target dimension table.

Auto CDC with DLT

```
-- DLT Auto CDC pattern for Slowly Changing Dimensions
APPLY CHANGES INTO uber_project.gold_layer.dimLocation
FROM STREAM(uber_project.silver_layer.obt)
KEYS (location_id)
SEQUENCE BY updated_at
STORED AS SCD TYPE 2
```

Orchestration & Scheduling

15-Minute Trigger

- Set up an automated Trigger in Databricks
- Pipeline runs every 15 minutes automatically
- Ensures near-real-time data freshness across all layers

Jobs & Notebooks

- All notebooks connected in a Databricks Job
- Execution order: Bronze notebooks → Silver notebooks → Gold notebooks
- Job tested and executed end-to-end successfully

Full End-to-End Run Order
1. User clicks 'Book' in WebApp → ride event sent to uber-eventhub-pro / ubertopic
2. ADF Pipeline (httpToStorageAccount): GitHub files → ADLS Gen2 (githubdata container)
3. Databricks Job — Bronze DLT: loads rides_raw from Event Hub + bulk_rides from ADLS
4. Silver DLT: merges streams, schema transforms, Jinja joins → OBT
5. Gold DLT: dimLocation (SCD2), dimPassenger, dimDriver, dimBooking, dimPayment, FactRides
6. Trigger repeats every 15 minutes for incremental updates

Quick Revision — Key Concepts

One-Liner Cheat Sheet

Concept	One-Line Summary
Event Hubs PaaS	No infra management — just use the service. Azure handles scaling, patching, hosting.
Metadata-Driven ADF	Config file drives pipeline — add new files without touching code.
Streaming Table (DLT)	Stateful bookmark — incremental merge, no re-scanning old data on each run.
Jinja2 Template	Dynamic SQL generation — add new joins via config, not code changes.
Watermark	Tells Spark when to drop old state from memory. Formula: Max Event Time - Lateness Delay.
SCD Type 2	Tracks dimension history — old records kept, new records inserted with sequence key.
Auto CDC (DLT)	APPLY CHANGES INTO — DLT manages SCD logic automatically.
SAS Token	Scoped, time-limited Azure Storage access — safer than Access Key.
STREAM keyword (SQL)	Tells DLT SQL to read table as streaming — equivalent to readStream in PySpark.
abfss:// protocol	Azure Blob File System Secure — Spark driver for ADLS Gen2 with SSL/TLS.

Top Interview Questions

Q1: When would you use Event Hubs over Service Bus?

Event Hubs: High-volume streaming (telemetry, IoT, clickstreams — millions/sec)

Service Bus: Transactional messaging needing sessions, dead-letter queues, ordering

Q2: What is a metadata-driven pipeline?

Pipeline driven by config/metadata — not hardcoded logic.

Add new sources by updating config only. No code changes needed.

Q3: Why use Streaming Tables instead of simple union/append?

Simple union re-reads ALL data every run — wastes compute, risks duplication on failure.

Streaming Table checkpoints state — only processes new data each run.

Q4: What is Watermarking and why is it needed?

Handles late-arriving streaming data. Tells Spark when to clear state from memory.

Without it: Spark keeps ALL windows in memory → OutOfMemory error.

Q5: What is SCD Type 2?

Tracks history of changes to dimension attributes.

Old record kept (with end date), new record inserted. Used in dimLocation here.